# Agentic Hub

*Production-Grade Planner + Supervisor Development, Design, Technical Approach, HLD/LLD, and Versioned Delivery Plan*

Prepared for: DD | Scope owner: Planner + Supervisor Layer | Version: 1.0

| Field | Value |
|---|---|
| Document purpose | Define a production-grade approach for designing and developing the Planner + Supervisor layer of Agentic Hub, while keeping interfaces, observability, governance hooks, and future extensibility aligned to the full enterprise platform. |
| Current owned scope | Planner, Supervisor, their contracts, decision logic, versioned delivery roadmap, production HLD/LLD, and minimal runtime handshake with Orchestrator. |
| Platform context | Agentic Hub will eventually include orchestration, memory, reflection, adaptive learning, guardrails, observability, governance, and domain agents; this document focuses only on the Planner + Supervisor layer and its immediate dependencies. |
| Development posture | Production-grade from day one: contract-first, observable, versioned, extensible, governed, and designed for rollout rather than one-off demo success. |

## 1. Executive Summary

The Planner + Supervisor layer is the decision core of Agentic Hub. The Planner transforms enterprise intent into a structured execution universe, and the Supervisor monitors whether execution remains valid, safe, useful, and within policy, cost, and reliability envelopes. In the full platform, the Orchestrator executes the plan, the Policy Plane constrains what may happen, Memory and Reflection improve future decisions, and Observability makes every important action traceable. This document upgrades the earlier Layer C POC plan into a production-grade, versioned development program while preserving the strong parts of the original approach: Planner-first sequencing, contract-first development, clear Planner/Supervisor/Orchestrator boundaries, realistic timelines, and explicit buffers.

Success for this workstream is not defined as “build the whole Agentic Hub.” It is defined as delivering a production-grade Planner + Supervisor service pair that can be onboarded into Agentic Hub as reusable platform services, support versioned enterprise workflows, integrate cleanly with policy, telemetry, and orchestration layers, and evolve through controlled versions from governed core runtime to reflection-ready and feedback-aware operation.

## 2. Scope, Ownership, and Boundaries

### 2.1 In-scope responsibilities

- Planner service design and implementation: intent normalization, task decomposition, DAG generation, plan validation, plan versioning, re-planning, and planning telemetry.
- Supervisor service design and implementation: state evaluation, decision emission, confidence and failure monitoring, escalation, stop/re-plan signaling, and supervisory telemetry.
- Canonical contracts for planning and supervision: RequestEnvelope, ExecutionPlan, TaskNode, StateCheckpoint, SupervisorDecision, ReplanRequest, and supporting enums/errors.
- Minimal runtime handshake with the Orchestrator so the Planner and Supervisor can operate in a real execution loop.
- Production hooks for policy, memory, reflection, evaluation, and observability so the layer remains compatible with the broader Agentic Hub roadmap.

### 2.2 Explicitly out of direct ownership in this document

- Full policy engine implementation and enterprise identity integration, although the layer must expose the hooks needed to consume those decisions.
- The complete long-term memory platform, although the Planner and Supervisor must be memory-aware by interface.
- Independent Reflection Agent and adaptive learning engine, although the Supervisor and Planner must emit reflection-ready artifacts.
- All domain agents and enterprise tool connectors, except for the runtime assumptions and contracts needed to call them.
- Platform-wide multi-tenancy, self-service onboarding, and org-wide operating model beyond what the Planner/Supervisor layer needs to integrate cleanly.

### 2.3 Boundary model

| Component | Primary responsibility | Owned in this workstream? | Notes |
|---|---|---|---|
| Planner | Define what should happen through structured plans, versions, and re-plans. | Yes | Primary design and build focus. |
| Supervisor | Monitor whether execution remains valid, safe, and useful; emit intervention decisions. | Yes | Primary design and build focus. |
| Orchestrator | Execute plan nodes, coordinate tools/agents, update runtime state. | Handshake only | Must remain distinct from Supervisor. |
| Policy plane | Identity, entitlements, action policy, approval policy, budget ceilings. | No | Consumed through hooks/contracts. |
| Memory and reflection | Historical state, procedural guidance, reflection, evaluation, learning. | Hooks only | Must be future-compatible. |
| Observability platform | Traces, metrics, logs, dashboards, incident | Partial | This layer must emit the right events. |

| Component | Primary responsibility | Owned in this workstream? | Notes |
|---|---|---|---|
| | telemetry. | | |

## 3. Production Design Principles

- Planner first, Supervisor second, integration third — but all designed as durable platform services rather than POC utilities.
- Contract-first development — every major boundary is represented as a typed schema, versioned and validated.
- Separation of cognition and control — models reason, decompose, summarize, and critique; deterministic systems own permissions, thresholds, budgets, auditing, and irreversible transitions.
- Planner output is data, not executable code — plans are structured artifacts to be validated, versioned, inspected, and replayed.
- Supervisor does not execute — it evaluates execution health and emits continue / reflect / re-plan / escalate / stop decisions. The Orchestrator remains the execution engine.
- Production observability by default — plan creation, state transitions, decision points, and exceptions must all be traceable.
- Model/provider abstraction — all model calls remain behind adapters so the core Planner and Supervisor logic survives provider changes.
- Versioned delivery — the layer will mature through planned versions rather than through uncontrolled incremental drift.
- Buffers are real work enablers — prompt tuning, schema refinement, integration instability, and confidence calibration always take time in systems like this.

## 4. Versioned Development Roadmap for Planner + Supervisor

The Planner + Supervisor layer will be delivered as a series of production versions. Each version expands capability while maintaining backward-compatible contracts and explicit release exit criteria.

| Version | Primary outcome | Planner + Supervisor scope | Buffered duration |
|---|---|---|---|
| V0.9 | Architecture baseline | Freeze boundaries, canonical contracts, versioning model, and HLD/LLD baselines. | 4–6 weeks |
| V1.0 | Governed Layer C core | Deliver production-grade Planner and Supervisor core runtime with minimal Orchestrator handshake. | 8–12 weeks |
| V1.5 | Trust hardening | Add policy-aware planning/supervision, approval checkpoints, stronger telemetry, and governance-ready escalation paths. | 6–10 weeks |
| V2.0 | Reflection-ready layer | Emit reflection- and evaluation-ready artifacts, structured post-run review payloads, and | 8–12 weeks |

| Version | Primary outcome | Planner + Supervisor scope | Buffered duration |
|---|---|---|---|
| | | supervisor evaluation signals. | |
| V2.5 | Memory + feedback aware | Use procedural/episodic memory hooks and human feedback signals to improve plan quality and supervisory decisions. | 8–10 weeks |
| V3.0 | Adaptive optimized layer | Introduce offline optimization hooks, candidate plan ranking, calibration, and shadow-evaluable decision policies. | 10–14 weeks |

Recommended planning assumption: one technical lead/architect, two strong engineers, and part-time platform/DevOps support. With that team shape, the full V0.9 → V3.0 journey is realistically a multi-quarter program rather than a short sprint series.

## 5. Target Position of Planner + Supervisor Inside Agentic Hub

The broader Agentic Hub architecture remains intact. The Planner + Supervisor layer sits in the control plane but must interoperate with the execution plane and policy plane. The Planner consumes request context, policies, available agent/tool metadata, and memory hints to create plans. The Orchestrator executes those plans. The Supervisor monitors plan execution, confidence, policy fit, budget use, and failure conditions, then emits intervention decisions.

### 5.1 Reference interaction model

1. A request arrives through API, UI, or event trigger.
2. Identity, risk, and policy context are resolved by adjacent layers.
3. The Planner receives a normalized RequestEnvelope plus policy/context metadata and generates an ExecutionPlan.
4. The Orchestrator executes ready TaskNodes, invoking tools and specialist agents and updating StateCheckpoint.
5. The Supervisor evaluates state at configured checkpoints and emits continue / reflect / re-plan / escalate / stop.
6. If re-plan is required, the Planner receives the current state and failure context and emits a new plan version.
7. Trace, evaluation, and audit artifacts are emitted throughout the lifecycle.
8. Reflection, memory curation, and adaptive learning consume run artifacts later through governed pipelines.

### 5.2 Production adjacency map

| Adjacent layer | What Planner consumes or emits | What Supervisor consumes or emits |
|---|---|---|
| Identity / Risk / Policy intake | Consumes role, entitlements, data sensitivity, risk class, autonomy level, budgets. | Consumes effective policy context, thresholds, approvals needed, autonomy constraints. |
| Orchestrator | Emits ExecutionPlan and revised plans. | Consumes state snapshots and emits structured decisions back into runtime. |

| Adjacent layer | What Planner consumes or emits | What Supervisor consumes or emits |
|---|---|---|
| Memory | Consumes procedural templates, episodic examples, and optionally semantic hints in later versions. | Consumes historical patterns and emits curated episodic signals in later versions. |
| Reflection / Evaluation | Emits plan quality metadata and failure context. | Emits decision rationales, failure summaries, and reflection-ready artifacts. |
| Observability / Audit | Emits plan version, node count, validation results, prompt/template versions. | Emits decision type, trigger reason, thresholds crossed, and intervention metadata. |

## 6. High-Level Design (HLD) — Planner

### 6.1 Planner mission

Transform enterprise intent into a validated, versioned, execution-ready plan that the Orchestrator can run and the Supervisor can monitor. The Planner defines the execution universe: tasks, dependencies, evidence requirements, execution modes, risk tags, timeouts, and re-planning paths.

### 6.2 Planner inputs

- RequestEnvelope: user or system intent, request metadata, channel, tenant, target outcome, budgets, and constraints.
- Policy and role context: entitlements, autonomy tier, approval requirements, data sensitivity, restricted tools, and residency constraints.
- Available execution surface: specialist agent capability profiles, tool contracts, model capability profiles, and fallback options.
- Memory/context hints (later versions): procedural templates, episodic examples, curated policy cues, and similar workflow histories.
- Failure context for replanning: current state snapshot, failed node, failure type, partial outputs, and supervisor trigger reason.

### 6.3 Planner outputs

- ExecutionPlan with globally unique plan_id and plan_version.
- Task DAG containing TaskNodes, dependencies, owner agent types, input/output contracts, evidence requirements, timeout/retry rules, rollback hints, and approval checkpoints where needed.
- Planning metadata: confidence, planning mode used, template or prompt version, validation status, and policy filters applied.
- Replan linkage: link to prior plan version and structured reason for re-planning.

### 6.4 Planner modes by version

- V1.0: deterministic templates + LLM-assisted DAG generation.
- V1.5: policy-aware planning with explicit approval/checkpoint insertion.
- V2.0: plan critique hooks and alternate-plan comparison.
- V2.5: memory-assisted planning using curated templates and prior trajectories.
- V3.0: candidate plan ranking, calibration, and optional simulation-assisted plan search.

### 6.5 Planner non-functional expectations

- Schema-safe outputs with deterministic validation.
- Predictable latency bands for first-pass planning and replanning.
- Replayability: same input context should yield inspectable and comparable outputs even when model stochasticity exists.
- Versionability: plans, templates, prompt versions, and validators are all versioned.
- Auditability: plan artifacts are logged without exposing unnecessary sensitive intermediate data.

## 7. Low-Level Design (LLD) — Planner

### 7.1 Planner component breakdown

| Sub-component | Responsibility | Initial technology choice |
|---|---|---|
| Planner API | Expose generate_plan() and replan() endpoints. | FastAPI + Pydantic |
| Intent normalizer | Normalize request constraints and effective planning context. | Python service module |
| Template registry | Store deterministic templates and reusable planning patterns. | YAML/JSON config + version control |
| LLM planner adapter | Call provider for schema-constrained DAG emission. | Provider adapter behind model gateway |
| Plan normalizer | Normalize provider output into canonical schema. | Python transformation layer |
| Plan validator | Check schema, node completeness, uniqueness, acyclic graph, and planning invariants. | Pydantic + NetworkX |
| Plan store / artifact emitter | Persist plan artifact, version metadata, validation output, and traces. | Artifact store + telemetry hooks |

### 7.2 Canonical contracts for Planner

```
{
  "RequestEnvelope": {"request_id": "...", "tenant_id": "...", "goal": "...", "risk_class":
"medium", "constraints": {"max_cost": 0, "max_latency_ms": 0}},
  "ExecutionPlan": {"plan_id": "...", "plan_version": 1, "planning_mode": "llm_structured",
"tasks": [...]},
  "TaskNode": {"node_id": "T1", "objective": "...", "owner_agent_type": "research_agent",
"dependencies": [], "evidence_requirements": [], "timeout_s": 30, "retry_policy": "limited",
"risk_level": "low"},
  "ReplanRequest": {"prior_plan_id": "...", "prior_plan_version": 1, "failed_node_id": "T2",
"failure_type": "tool_timeout", "state_snapshot_ref": "..."}
}
```

### 7.3 Planner execution flow

9. Receive RequestEnvelope and effective planning context.
10. Choose planning mode: deterministic template, assisted LLM planning, or later policy/memory-aware path.
11. Generate candidate plan artifact.
12. Normalize provider output into canonical schema.
13. Run structural validation: schema, required fields, node uniqueness, dependency validity, DAG acyclicity, and contract presence.
14. Run semantic validation: unsupported agent type, missing evidence requirements, invalid approval placement, impossible dependency pattern, or policy mismatch.

15. If validation fails, run constrained retry/repair loop; if still invalid, return structured planning failure.
16. Persist plan artifact and emit telemetry.

### 7.4 Planner API surface (recommended)

```
POST /planner/generate
POST /planner/replan
GET  /planner/plan/{plan_id}/{version}
GET  /planner/health
GET  /planner/contracts
```

### 7.5 Planner telemetry events

- plan.generate.started / completed / failed
- plan.validation.failed with failure category
- plan.replan.started / completed
- plan.node.count, planning_mode, prompt_or_template_version, plan_version
- plan.policy.filtered and plan.approval.checkpoints.inserted in later versions

## 8. High-Level Design (HLD) — Supervisor

### 8.1 Supervisor mission

The Supervisor ensures that execution remains valid, safe, useful, and aligned to cost, risk, and quality constraints. It does not execute the workflow; it evaluates execution state and decides whether the runtime should continue, reflect, re-plan, escalate, or stop.

### 8.2 Supervisor inputs

- Current StateCheckpoint and current plan version.
- Task results, errors, confidence signals, tool statuses, and budget consumption.
- Policy and risk envelope effective for the running request.
- Optional memory and reflection signals in later versions.
- Operator or human approval state when applicable.

### 8.3 Supervisor outputs

- SupervisorDecision with decision_type, trigger_reason, decision_confidence, and next recommended action.
- ReplanRequest when re-planning is necessary.
- Escalation payload with state snapshot, failure summary, trace reference, and policy metadata.
- Reflection/evaluation artifact pointers in later versions.

### 8.4 Supervisor decision set

- continue
- reflect
- re-plan
- escalate
- stop

### 8.5 Supervisor non-functional expectations

- Deterministic and explainable baseline behavior in early versions.
- Fast enough to evaluate at runtime checkpoints without becoming a bottleneck.
- Traceability of why a decision was produced.

- Strict respect for policy-plane ceilings and non-negotiable constraints.
- Safe evolution path toward better calibration and offline optimization later.

## 9. Low-Level Design (LLD) — Supervisor

### 9.1 Supervisor component breakdown

| Sub-component | Responsibility | Initial technology choice |
|---|---|---|
| Supervisor API | Expose evaluate_state() and escalation-oriented endpoints. | FastAPI + Pydantic |
| State reader | Read and normalize StateCheckpoint + plan/task metadata. | Python service module |
| Signal extractor | Compute runtime signals: confidence, failures, thresholds crossed, retries, cost/latency state. | Python rules module |
| Conflict detector | Detect contradictions, repeated failures, unresolved dependencies, invalid task outcomes. | Rule-based evaluator |
| Decision engine | Emit continue / reflect / re-plan / escalate / stop. | Config-driven deterministic engine |
| Escalation composer | Prepare operator-readable escalation payload. | Python formatter + artifact refs |
| Reflection hook emitter | Emit structured reflection/evaluation payloads for later versions. | Artifact/telemetry layer |

### 9.2 Canonical contracts for Supervisor

```
{
  "StateCheckpoint": {"plan_id": "...", "plan_version": 2, "task_statuses": {"T1":
"success", "T2": "failed"}, "budget": {"cost": 0.0, "latency_ms": 0}, "signals":
{"confidence": 0.72}},
  "SupervisorDecision": {"decision_type": "re-plan", "trigger_reason": "task_timeout",
"decision_confidence": 0.84, "requires_human": false, "next_action": "planner.replan"},
  "EscalationPayload": {"severity": "high", "summary": "Repeated task timeout", "trace_ref":
"...", "state_snapshot_ref": "..."}
}
```

### 9.3 Supervisor evaluation pipeline

17. Read current StateCheckpoint and associated ExecutionPlan metadata.
18. Extract relevant signals: failed nodes, unresolved critical dependencies, budget exhaustion, confidence degradation, repeated retries, policy-blocked nodes.
19. Run conflict and anomaly checks.
20. Apply decision policy thresholds and policy-plane hard constraints.
21. Emit SupervisorDecision and optional ReplanRequest or EscalationPayload.
22. Persist decision telemetry and attach rationale summary.

### 9.4 Supervisor API surface (recommended)

```
POST /supervisor/evaluate
POST /supervisor/escalate
GET  /supervisor/decision/{request_id}
GET  /supervisor/health
GET  /supervisor/contracts
```

### 9.5 Supervisor telemetry events

- supervisor.evaluate.started / completed / failed
- supervisor.decision.emitted with decision_type and reason category
- supervisor.replan.requested
- supervisor.escalation.created
- supervisor.threshold.crossed for budget, confidence, policy, and reliability dimensions

## 10. Shared Runtime, Deployment, and Technical Stack

### 10.1 Recommended baseline stack

| Area | Baseline choice | Why it fits production-oriented development | Upgrade path |
|---|---|---|---|
| Service/API layer | FastAPI | Typed APIs, OpenAPI contracts, good fit for contract-first service design. | Can remain long-term or sit behind gateway/service mesh. |
| Schema/contracts | Pydantic v2 | Strong typed validation, JSON Schema export, versionable models. | Schema registry / protobuf later if needed cross-language. |
| Planner structured outputs | Managed LLM endpoint with schema-constrained output | Reduces JSON drift and accelerates Planner stabilization. | Private or local model through same adapter later. |
| DAG validation | NetworkX | Reliable acyclic checks and graph utilities. | Custom optimized validator later if scale needs it. |
| Runtime orchestration handshake | LangGraph or equivalent stateful graph runtime | Natural fit for plan → execute → supervise → re-plan loops. | Custom state machine or durable workflow engine later. |
| Observability | OpenTelemetry + structured logs | Portable production telemetry foundation. | Attach Phoenix/MLflow/enterprise APM later. |
| Packaging/deploy | Docker + Azure Container Apps (or equivalent) | Clean path from local dev to test/prod service deployment. | Kubernetes/service mesh if platform broadens. |

### 10.2 Repository/module structure

```
api/
  planner_api.py
  supervisor_api.py
core/
  contracts/
  planner/
  supervisor/
  orchestrator_handshake/
  config/
  telemetry/
adapters/
  llm/
  tools/
  policy/
  memory/
tests/
  unit/
  integration/
  contract/
```

```
  performance/
prompts/
configs/
```

### 10.3 Deployment topology assumption

- Dev: local service containers, local config, mocked tools, contract tests, trace export to local collector.
- Test: containerized Planner and Supervisor services with shared model gateway and mocked or controlled tool endpoints.
- Prod-like staging: policy hooks enabled, real observability wiring, replayable artifacts, controlled test workflows, and non-production data.

## 11. Phase-by-Phase Production Development Plan

The earlier POC phases remain useful, but each phase is now upgraded to production-grade expectations: stronger contracts, compatibility with policy and telemetry, stricter exit criteria, and more realistic platform readiness assumptions.

### Version V0.9 — Architecture Baseline

Duration: 4–6 weeks total (3–4 weeks build + 1–2 weeks buffer)

#### HLD focus

- Freeze Planner/Supervisor/Orchestrator boundaries.
- Publish canonical schemas and versioning policy.
- Publish service contracts, sequence diagrams, and operating assumptions.
- Define release gates, non-goals, and production quality bars.

#### LLD / build focus

- Define all canonical Pydantic models and enums.
- Define plan and decision versioning semantics.
- Define initial API surface, config model, error contracts, and trace event names.
- Initialize repository skeleton and contract test harness.

#### Exit outputs

- Approved HLD baseline
- Schema package with JSON Schema exports
- Initial service skeletons
- Architecture review sign-off

### Version V1.0 — Governed Layer C Core

Duration: 8–12 weeks total (6–9 weeks build + 2–3 weeks buffer)

#### HLD focus

- Deliver Planner core and Supervisor core as reusable services.
- Implement deterministic + LLM-assisted planning.
- Implement schema-safe validation, replanning, supervisory decision engine, and minimal runtime handshake.
- Instrument traces and structured logs across planning and supervision.

#### LLD / build focus

- Implement planner modes, validator, normalizer, prompt/template registry, and plan persistence.

- Implement supervisor state reader, signal extraction, deterministic decisions, escalation payloads, and reflection trigger placeholder.
- Wire minimal Orchestrator checkpoints and plan/state transitions.
- Deliver contract tests, unit tests, integration tests, and golden workflow demos.

#### Exit outputs

- Runnable end-to-end governed Layer C core
- Traceable planning and supervision runs
- Stable replanning flow
- Release readiness review for V1.0

### Version V1.5 — Trust Hardening

Duration: 6–10 weeks total (5–7 weeks build + 1–3 weeks buffer)

#### HLD focus

- Add policy-aware planning constraints.
- Insert approval checkpoints into plans where required.
- Make supervisory decisions risk-tier aware.
- Strengthen incident-grade escalation and auditability.

#### LLD / build focus

- Integrate policy context adapters.
- Add approval-checkpoint generation and policy-aware node tagging.
- Add calibrated escalation categories and richer rationale capture.
- Add replay/debug support for plan and decision history.

#### Exit outputs

- Policy-aware planner and supervisor
- Governance-ready audit artifacts
- Risk-tiered escalation behavior
- Staging sign-off for governed flows

### Version V2.0 — Reflection-Ready Layer

Duration: 8–12 weeks total (6–9 weeks build + 2–3 weeks buffer)

#### HLD focus

- Make Planner and Supervisor emit artifacts that Reflection/Evaluation services can consume.
- Add plan critique hooks, failure attribution hints, and post-run review payloads.
- Keep learning offline-governed, not live-self-mutating.

#### LLD / build focus

- Add structured reflection payloads from Planner and Supervisor.
- Add route-to-reflection-agent contract.
- Add plan quality scoring and supervisor evaluation signal emission.
- Add trajectory-linked post-run review objects.

#### Exit outputs

- Reflection-ready artifacts

- Evaluation-ready traces
- Failure-attribution hooks
- Offline learning compatibility

### Version V2.5 — Memory + Feedback Aware

Duration: 8–10 weeks total (6–8 weeks build + 2 weeks buffer)

#### HLD focus

- Use procedural and episodic memory hooks to improve planning and supervision.
- Allow human feedback to enrich decision and plan quality over time.
- Keep memory writes curated and governed.

#### LLD / build focus

- Integrate procedural template retrieval in Planner.
- Integrate episodic pattern signals in Supervisor.
- Add feedback-linked plan and decision metadata.
- Add curated memory write-back contract.

#### Exit outputs

- Memory-aware planning
- History-aware supervision
- Feedback-linked artifacts
- Governed memory write hooks

### Version V3.0 — Adaptive Optimized Layer

Duration: 10–14 weeks total (8–11 weeks build + 2–3 weeks buffer)

#### HLD focus

- Add offline optimization hooks for plan ranking and calibrated decision policies.
- Introduce candidate policy registries, shadow evaluation, and performance comparisons.
- Keep policy-plane hard boundaries deterministic.

#### LLD / build focus

- Add candidate-plan ranking and calibration interfaces.
- Add supervisor policy registry and offline scoring inputs.
- Add shadow-compatible evaluation fields.
- Add release/promotion hooks for improved policies.

#### Exit outputs

- Shadow-evaluable optimized Planner/Supervisor
- Offline optimization pipeline compatibility
- Promotion/rollback hooks
- Enterprise architecture review for next-stage rollout

## 12. Timeline Summary with Buffers

| Version | Core build | Buffer | Total | Realistic calendar interpretation |
|---|---|---|---|---|

| Version | Core build | Buffer | Total | Realistic calendar interpretation |
|---|---|---|---|---|
| V0.9 | 3–4 weeks | 1–2 weeks | 4–6 weeks | Freeze architecture, contracts, and readiness to build. |
| V1.0 | 6–9 weeks | 2–3 weeks | 8–12 weeks | Deliver first reusable governed Planner + Supervisor core. |
| V1.5 | 5–7 weeks | 1–3 weeks | 6–10 weeks | Add policy-aware hardening and governance-ready behavior. |
| V2.0 | 6–9 weeks | 2–3 weeks | 8–12 weeks | Emit reflection/evaluation-ready artifacts and hooks. |
| V2.5 | 6–8 weeks | 2 weeks | 8–10 weeks | Introduce memory and feedback awareness. |
| V3.0 | 8–11 weeks | 2–3 weeks | 10–14 weeks | Introduce offline optimization and shadow-ready policies. |

Practical planning guidance: if the team is only one lead plus one engineer, use the high end of each range. If the team is one lead plus two strong engineers plus reliable platform support, the midpoints are realistic. Do not remove buffers: structured-output drift, prompt tuning, policy-edge cases, calibration, and runtime debugging will consume them.

## 13. Risks, Constraints, and Mitigations

| Risk | Why it matters in production | Planned mitigation |
|---|---|---|
| Boundary confusion | Supervisor can accidentally absorb orchestration duties; Planner can start emitting executable logic instead of plan artifacts. | Freeze boundaries in V0.9; review code ownership against architecture every version. |
| Structured output drift | Planner outputs may be syntactically valid but semantically poor or incomplete. | Use constrained outputs, strict validators, golden workflow tests, and retry/repair loops. |
| Prompt tuning churn | Production planning quality often takes multiple prompt/template iterations. | Version prompts/templates; keep explicit buffers in V1.0 and V1.5. |
| Policy mismatch | Plans or supervisory decisions may violate enterprise rules if policy hooks are vague. | Make policy input explicit in contracts; keep hard policy checks outside the model. |
| Integration instability | Replanning and runtime checkpoint logic introduce hard-to-debug edge cases. | Add full trace coverage and replayability before calling a version stable. |
| Escalation quality | Too many false escalations reduce trust; missed escalations create risk. | Track both false and missed escalation metrics; calibrate by workflow tier. |
| Version sprawl | Unversioned prompt/config/contract changes | Version all contracts, prompts, templates, and threshold configs. |

| Risk | Why it matters in production | Planned mitigation |
|---|---|---|
| | create silent regressions. | |

## 14. Production Readiness Checklist for Planner + Supervisor

- Canonical contracts are versioned, reviewed, and published.
- Planner outputs are schema-valid, replayable, and auditable.
- Supervisor decisions are explainable and linked to state and threshold context.
- Orchestrator handshake is stable and Supervisor does not own execution.
- Policy context is explicitly represented in planning and supervision input contracts.
- Trace coverage exists for plan generation, validation failures, state transitions, decisions, and escalations.
- Golden workflow suite exists for representative enterprise use cases.
- Failure, timeout, and re-plan scenarios have been exercised and documented.
- Release notes document contract changes, prompt/template changes, threshold changes, and backward compatibility implications.

## 15. Immediate Next Steps

23. Finalize V0.9 scope and architecture review pack.
24. Create the versioned contract package and schema tests.
25. Stand up Planner and Supervisor service skeletons with health and contract endpoints.
26. Implement Planner deterministic path first, then structured-output path, then validator.
27. Implement Supervisor state reader, signal extraction, and deterministic decision engine.
28. Integrate minimal Orchestrator checkpoints only after Planner/Supervisor contracts are stable.
29. Create the Jira planning sheet aligned to these versions, features, and buffered timelines.

## Appendix A — Recommended Canonical Models

- RequestEnvelope
- ExecutionPlan
- TaskNode
- StateCheckpoint
- SupervisorDecision
- ReplanRequest
- EscalationPayload
- PlanValidationResult
- DecisionTelemetryEvent

## Appendix B — Suggested KPIs by version

- Planner: valid plan rate, replan success rate, average nodes per workflow, policy-checkpoint insertion correctness.
- Supervisor: false escalation rate, missed escalation rate, mean decision latency, recoverable failure handling rate.
- Shared: trace completeness, plan/deployment version traceability, integration stability, workflow completion rate in governed test cases.